Trabajo Práctico 5 – Sincronización

SISTEMAS OPERATIVOS

Índice

1. Comunicación entre procesos con tuberías **3**
1.1. Descriptores de archivo 3
1.2. Construir `ls | wc` 4
1.3. Tuberías con nombre: `mkfifo` 6

2. Exclusión mutua con candados de archivos **7**
2.1. Un candado bloqueante 7

3. Semáforos de System V **9**
3.1. Permisos del conjunto 10
3.2. Crear e inicializar un semáforo 10
3.3. Consultar y eliminar el conjunto 11

En este TP vas a estudiar cómo cooperan los procesos cuando deben comunicarse o coordinar el acceso a un recurso compartido. Vamos a usar llamadas POSIX (*Portable Operating System Interface*) para construir una tubería y candados de archivos, y la interfaz de IPC de System V para los semáforos. En todos los casos, observá primero qué hace el núcleo y después relacioná el resultado con los descriptores, los procesos y sus estados.

1. Comunicación entre procesos con tuberías

Una tubería o *pipe* es un canal unidireccional de comunicación entre procesos. Un proceso escribe una secuencia de bytes en un extremo y otro proceso la lee del extremo opuesto. Por ejemplo, el shell puede conectar `ls` con `wc` mediante el carácter `|`:

```
$ ls | wc
```

`wc` (*word count*) muestra, por defecto, la cantidad de líneas, palabras y bytes que recibió. El shell crea un `pipe`, conecta los descriptores estándar de cada proceso y luego ejecuta los programas.

1.1. Descriptores de archivo

Un descriptor de archivo es un número entero que identifica, dentro de un proceso, un archivo o canal abierto. Cada proceso comienza normalmente con tres descriptores estándar:

Valor	Nombre	Constante en C
0	Entrada estándar (`stdin`)	`STDIN_FILENO`
1	Salida estándar (`stdout`)	`STDOUT_FILENO`
2	Error estándar (`stderr`)	`STDERR_FILENO`

La llamada `pipe(fd)` recibe un arreglo de dos enteros. Si tiene éxito, `fd[0]` es el extremo de lectura y `fd[1]` el extremo de escritura. Un pipe anónimo no tiene un nombre en el sistema de archivos: los procesos pueden usarlo si heredan sus descriptores, normalmente después de un `fork()`. Una FIFO (*First In, First Out*), en cambio, tiene un nombre y permite que procesos no relacionados la abran mediante ese nombre.

El pipe es un canal gestionado por el núcleo, no un archivo temporal. El núcleo mantiene un búfer para el canal y entrega los bytes en el mismo orden en que fueron escritos. Si no queda ningún descriptor de escritura abierto, una lectura puede observar fin de archivo; si el búfer está vacío pero todavía existe un escritor, la lectura espera.

1.2. Construir `ls | wc`

El siguiente programa crea dos hijos. El primero ejecuta `ls` y redirige su salida al pipe; el segundo ejecuta `wc` y redirige su entrada desde el pipe. El proceso original cierra sus copias de ambos extremos y espera a los dos hijos.

```c
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int fd[2];
    pid_t productor;
    pid_t consumidor;
    if (pipe(fd) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }

    productor = fork();
    if (productor == -1) {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        return EXIT_FAILURE;
    }
    if (productor == 0) {
        close(fd[0]);
        if (dup2(fd[1], STDOUT_FILENO) == -1) {
            perror("dup2");
            _exit(EXIT_FAILURE);
        }
        close(fd[1]);
        execlp("ls", "ls", (char *)NULL);
        perror("ls");
        _exit(EXIT_FAILURE);
    }

    consumidor = fork();
    if (consumidor == -1) {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        waitpid(productor, NULL, 0);
        return EXIT_FAILURE;
    }
    if (consumidor == 0) {
        close(fd[1]);
        if (dup2(fd[0], STDIN_FILENO) == -1) {
            perror("dup2");
            _exit(EXIT_FAILURE);
        }
        close(fd[0]);
        execlp("wc", "wc", (char *)NULL);
        perror("wc");
        _exit(EXIT_FAILURE);
    }

    close(fd[0]);
    close(fd[1]);
    waitpid(productor, NULL, 0);
    waitpid(consumidor, NULL, 0);
    return EXIT_SUCCESS;
}
```

Copiá el listado anterior en un archivo llamado `tuberia.c`. Luego compilá y ejecutá el programa. La salida puede variar según los archivos del directorio actual:

```
$ gcc -Wall -Wextra -std=c11 -o tuberia tuberia.c
$ ./tuberia
     8 8 74
```

Los números mostrados son sólo un ejemplo. Probá también la versión del shell y compará las dos ejecuciones:

```
$ ls | wc
$ strace -f -e trace=pipe,pipe2,dup2,close,execve,wait4 ./tuberia
```

En el hijo que ejecuta `ls`, `dup2(fd[1], STDOUT_FILENO)` hace que el descriptor 1 apunte al extremo de escritura. En el otro hijo, la misma operación hace que el descriptor 0 apunte al extremo de lectura. Después de duplicar, cada proceso cierra los descriptores que ya no necesita. Esos `close()` son necesarios para que `wc` pueda detectar el fin de archivo. Si permanece abierto algún descriptor de escritura, la lectura puede quedar esperando.

EXTRA

1. Hacé un diagrama del pipe y marcá qué proceso conserva cada extremo después de los cierres. Indicá qué representan los descriptores 0 y 1 en cada hijo.
2. Explicá por qué el programa crea dos hijos en vez de ejecutar `wc` directamente en el proceso original.
3. Agregá una tercera etapa para construir una cadena equivalente a `ls | wc -l | cat`. ¿Qué extremos debe cerrar cada proceso?
4. Consultá `/proc/<pid>/fd` mientras un proceso permanece bloqueado en una lectura. Relacioná los enlaces simbólicos con los descriptores del programa.

1.3. Tuberías con nombre: `mkfifo`

Una FIFO tiene una entrada permanente en un directorio y puede ser abierta por procesos que no tienen un ancestro común. El nombre no contiene los datos: sólo permite que los procesos encuentren el mismo canal del núcleo. Abrirla para leer o escribir puede bloquearse hasta que exista un proceso en el extremo opuesto.

Probala desde dos terminales. En la primera terminal creá la FIFO y dejá un lector esperando:

```
$ mkfifo canal
$ ls -l canal
$ cat < canal
```

En la segunda terminal escribí un mensaje y observá que `cat` termina cuando recibe el fin de archivo:

```
$ printf 'mensaje enviado por una FIFO\n' > canal
```

La entrada `canal` permanece en el directorio aunque los procesos hayan terminado. Eliminá la FIFO cuando finalices la prueba:

```
$ rm canal
```

Repetí la prueba con `echo`, `wc` o dos programas propios. Compará la FIFO con el pipe anónimo del programa anterior: ¿qué propiedad permite que la usen procesos que no tienen un ancestro común?

EXTRA

Como extensión, inspeccioná los descriptores de los procesos mientras la FIFO está abierta con `ls -l /proc/<pid>/fd`. Relacioná los enlaces simbólicos con el nombre `canal` y con el estado bloqueado de la operación de apertura.

2. Exclusión mutua con candados de archivos

Cuando varios procesos acceden al mismo recurso, una operación puede necesitar exclusión mutua: mientras un proceso está en su sección crítica, los demás deben esperar. Sin coordinación, dos procesos pueden leer el mismo estado, calcular resultados incompatibles y sobrescribir los cambios del otro.

`lockf()` permite solicitar un candado exclusivo sobre una región de un archivo abierto. El tamaño se cuenta desde la posición actual del descriptor. Las operaciones principales son:

Operación	Comportamiento
`F_LOCK`	solicita el candado y espera si la región está ocupada
`F_TLOCK`	solicita el candado y falla inmediatamente si está ocupado
`F_TEST`	comprueba si la región está disponible, sin tomarla
`F_ULOCK`	libera el candado de la región

Estos candados son normalmente *advisory*. El núcleo coordina únicamente a los procesos que intentan adquirir el candado; un programa que lo ignore puede escribir igualmente en el archivo. El candado se libera al ejecutar `F_ULOCK` o cuando el proceso cierra el archivo o termina.

2.1. Un candado bloqueante

El ejemplo toma el candado de los primeros 100 bytes durante cinco segundos. El argumento opcional permite que varias copias usen el mismo

archivo. La llamada a `ftruncate()` deja preparada la región, y `lseek()` coloca la posición al comienzo antes de llamar a `lockf()`.

```c
#define _XOPEN_SOURCE 700

#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    const char *ruta = argc == 2 ? argv[1] : "archivo";
    int fd = open(ruta, O_RDWR | O_CREAT, 0666);

    if (fd == -1) {
        perror("open");
        return EXIT_FAILURE;
    }
    if (ftruncate(fd, 100) == -1) {
        perror("ftruncate");
        close(fd);
        return EXIT_FAILURE;
    }
    if (lseek(fd, 0, SEEK_SET) == (off_t)-1) {
        perror("lseek");
        close(fd);
        return EXIT_FAILURE;
    }

    printf("Proceso %ld: esperando el candado\n", (long)getpid());
    fflush(stdout);
    if (lockf(fd, F_LOCK, 100) == -1) {
        perror("lockf");
        close(fd);
        return EXIT_FAILURE;
    }

    printf("Proceso %ld: candado colocado\n", (long)getpid());
    fflush(stdout);
    sleep(5);
    if (lockf(fd, F_ULOCK, 100) == -1) {
        perror("lockf");
        close(fd);
        return EXIT_FAILURE;
    }
    printf("Proceso %ld: candado eliminado\n", (long)getpid());

    close(fd);
    return EXIT_SUCCESS;
}
```

Copiá el listado anterior en un archivo llamado `candado.c`. Compilá el

archivo y ejecutá primero una sola copia:

```
$ gcc -Wall -Wextra -std=c11 -o candado candado.c
$ ./candado archivo.bin
Proceso 2659: esperando el candado
Proceso 2659: candado colocado
Proceso 2659: candado eliminado
```

Después iniciá tres copias sobre el mismo archivo y esperá a que todas terminen:

```
$ ./candado archivo.bin & p1=$!
$ ./candado archivo.bin & p2=$!
$ ./candado archivo.bin & p3=$!
$ wait "$p1" "$p2" "$p3"
```

Sólo un proceso debería mostrar “candado colocado” a la vez. El orden de adquisición no tiene por qué coincidir con el orden de lanzamiento ni con el valor de los PID. Mientras el experimento está corriendo, observá `/proc/locks` desde otra terminal:

```
$ cat /proc/locks
```

Una línea puede indicar la clase del candado (por ejemplo, `POSIX`), si es asesor o obligatorio, el modo de acceso, el PID, el dispositivo e inode y los límites de la región. La representación y la cantidad de líneas dependen de la versión del núcleo y de los demás procesos del sistema.

1. Registrá para cada copia el PID, el instante en que empezó a esperar, el instante en que obtuvo el candado y el instante en que lo liberó.
2. ¿Qué proceso entra primero? ¿Podés deducir una política de planificación a partir de ese orden? Justificá la respuesta.
3. Cambiá `F_LOCK` por `F_TLOCK`. ¿Qué diferencia observás y qué código de error deberías tratar en un programa real?
4. ¿Por qué este mecanismo no evita que un programa que use `write()` sin llamar a `lockf()` modifique el archivo?

EXTRA

Investigá `flock()` y los candados POSIX de `fcntl()`. Compará qué objetos identifican, qué ocurre al duplicar un descriptor y si los candados son heredados después de `fork()`. Consultá las páginas de manual de `flock(2)`, `fcntl(2)` y `lockf(3)`.

3. Semáforos de System V

Un semáforo es un contador mantenido por el núcleo para coordinar el acceso a un recurso. En el caso binario, el valor 1 puede representar “disponible” y el valor 0 “ocupado”. La operación de espera decrementa el contador y

se bloquea si no puede hacerlo; la operación de señal lo incrementa cuando el proceso libera el recurso. A diferencia de un pipe, el semáforo no transporta datos: sólo sincroniza.

En este apartado usaremos la interfaz de IPC (*Inter-Process Communication*) de System V:

- `semget()` crea un conjunto de semáforos o devuelve el identificador de uno existente;
- `semop()` aplica una o más operaciones atómicas descritas por estructuras `sembuf`;
- `semctl()` consulta, modifica o elimina el conjunto.

3.1. Permisos del conjunto

Los permisos de un semáforo de System V se aplican al conjunto completo y siguen el esquema de propietario, grupo y otros usuarios. En este caso se habla de lectura y alteración: el conjunto no contiene un flujo de datos como un archivo.

- El permiso de lectura permite consultar valores y metadatos, por ejemplo con `GETVAL`, `GETALL` o `IPC_STAT`.
- El permiso de alteración permite cambiar valores con `semop()`, `SETVAL` o `SETALL`.
- `IPC_SET` e `IPC_RMID` requieren ser propietario o creador del conjunto, o tener privilegios suficientes.

Por ejemplo, `0660` permite leer y alterar el conjunto al propietario y al grupo, pero no da acceso a otros usuarios. En la columna `perms` de `ipcs -s`, el bit que en un archivo se suele llamar “escritura” representa aquí alteración.

Una estructura `sembuf` indica el índice del semáforo, el cambio que se quiere hacer en `sem_op` y opciones como `IPC_NOWAIT`. Una operación positiva incrementa el valor; una operación negativa espera hasta que haya suficiente valor y luego lo decrementa. Una operación cero espera hasta que el valor llegue a cero.

3.2. Crear e inicializar un semáforo

El programa siguiente crea un conjunto con un semáforo. La llamada usa `IPC_PRIVATE`. Esta constante indica a `semget()` que debe crear un conjunto nuevo; no restringe su uso al proceso creador. El programa incrementa el semáforo a 1 y deja el conjunto existente para que puedas inspeccionarlo y eliminarlo en el paso siguiente.

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/sem.h>

int main(void) {
    struct sembuf incrementar = {
        .sem_num = 0,
        .sem_op = 1,
        .sem_flg = 0,
    };
    int semid = semget(IPC_PRIVATE, 1, 0666);

    if (semid == -1) {
        perror("semget");
        return EXIT_FAILURE;
    }
    if (semop(semid, &incrementar, 1) == -1) {
        perror("semop");
        semctl(semid, 0, IPC_RMID);
        return EXIT_FAILURE;
    }

    printf("Conjunto creado: %d\n", semid);
    printf("Valor inicial: 1 (recurso disponible).\n");
    printf("Eliminar con: ./cntrlsem %d\n", semid);
    return EXIT_SUCCESS;
}
```

Copiá el listado anterior en un archivo llamado `creasem.c`. Compilá el archivo y compará la lista de semáforos antes y después de ejecutar:

```
$ gcc -Wall -Wextra -std=c11 -o creasem creasem.c
$ ipcs -s
$ ./creasem
Conjunto creado: 98307
Valor inicial: 1 (recurso disponible).
Eliminar con: ./cntrlsem 98307
$ ipcs -s
```

El identificador del ejemplo cambia en cada ejecución. Guardá el valor que imprima tu sistema; no copies `98307`. Observá las columnas de `ipcs`: clave, identificador, propietario, permisos y cantidad de semáforos del conjunto.

3.3. Consultar y eliminar el conjunto

El comando `semctl()` recibe el identificador del conjunto y el índice del semáforo. En este TP usamos un conjunto de un elemento, por eso el índice es siempre cero. Algunas operaciones útiles son:

Comando	Efecto
`GETVAL`	devuelve el valor actual del semáforo
`SETVAL`	establece el valor indicado por el argumento
`GETPID`	devuelve el PID del último proceso que ejecutó `semop()`
`GETNCNT`	devuelve cuántos procesos esperan que el valor aumente
`GETZCNT`	devuelve cuántos procesos esperan que el valor sea cero
`GETALL`	devuelve los valores de todos los semáforos del conjunto
`SETALL`	establece los valores de todos los semáforos del conjunto
`IPC_SET`	modifica propietario, grupo y permisos del conjunto
`IPC_STAT`	copia la configuración del conjunto a una estructura `semid_ds`
`IPC_RMID`	elimina el conjunto de semáforos

`GETALL` y `SETALL` usan un arreglo dentro del argumento de `semctl()`; `IPC_STAT` y `IPC_SET` usan una estructura `semid_ds`. En todos los casos, el proceso debe tener los permisos necesarios para consultar o modificar el conjunto.

El programa de control usa `IPC_RMID`:

```c
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/sem.h>

int main(int argc, char *argv[]) {
    char *final;
    long valor;

    if (argc != 2) {
        fprintf(stderr, "Uso: %s SEMID\n", argv[0]);
        return EXIT_FAILURE;
    }

    errno = 0;
    valor = strtol(argv[1], &final, 10);
    if (errno != 0 || final == argv[1] || *final != '\0' || valor < 0 ||
        valor > INT_MAX) {
        fprintf(stderr, "SEMID invalido: %s\n", argv[1]);
        return EXIT_FAILURE;
    }
    if (semctl((int)valor, 0, IPC_RMID) == -1) {
        perror("semctl IPC_RMID");
        return EXIT_FAILURE;
    }

    puts("Conjunto eliminado.");
    return EXIT_SUCCESS;
}
```

Copiá el listado anterior en un archivo llamado `cntrlsem.c` y compilá el archivo:

```
$ gcc -Wall -Wextra -std=c11 -o cntrlsem cntrlsem.c
```

Pasá el identificador que imprimió `creasem` y verificá que ya no aparezca en `ipcs -s`:

```
$ ./cntrlsem 98307
Conjunto eliminado.
$ ipcs -s
```

Si cerraste la terminal antes de eliminar el conjunto, todavía podés quitarlo con `ipcrm -s SEMID`, siempre que tengas permisos. Los semáforos de System V son recursos persistentes del núcleo: terminar el programa creador no los elimina automáticamente.

EXTRA

1. Ejecutá `creasem` dos veces sin eliminar los conjuntos. ¿Cuántas entradas nuevas aparecen y por qué `IPC_PRIVATE` no hace que ambas ejecuciones compartan el mismo conjunto?
2. Eliminá los dos conjuntos y repetí la observación con `ipcs -s`. ¿Qué diferencia hay entre el valor del semáforo y el identificador del conjunto?
3. Relacioná la operación negativa de `semop()` con la espera bloqueante de `lockf(F_LOCK)`. ¿Qué tienen en común y qué no protegen por sí solos?
4. Consultá `/proc/sysvipc/sem` y compará su información con `ipcs -s`.